# Reto 2 — La Caja Negra

**Reemplazo byte-equivalente de un servicio legado · Documento de diseño**

Autor: Angel Mendieta · Equipo: _____ · 10 de octubre de 2026

---

Este documento describe el diseño del reemplazo para el Reto 2. El servicio original es una caja negra de 10 años en producción, sin código, documentación ni autores disponibles. El objetivo es entender su comportamiento por completo mediante experimentación sistemática y construir un reemplazo que responda de forma idéntica —byte a byte— a cualquier entrada, incluyendo errores, rutas inexistentes, autenticación y estado interno.

## 1. Problema y objetivo

La evaluación se realiza sobre un conjunto oculto de pruebas que ejercita deliberadamente casos límite, por lo que reproducir únicamente el "camino feliz" es insuficiente. El reemplazo debe:

- Aceptar exactamente los mismos requests que el original, incluidas las rutas que no existen, y devolver las mismas respuestas de error.
- Replicar el comportamiento de autenticación.
- Modelar el estado interno y poder reiniciarlo mediante `POST /__reset`.
- No llamar nunca al servicio original ni a otro reemplazo.

## 2. Principio de diseño: separar comportamiento de transporte

El comportamiento del servicio se implementa como una función pura, `respond(req, state) → (status, headers, body, state)`, que no depende de red ni de ningún framework. El servidor —una aplicación ASGI mínima— actúa solo como transporte de bytes: lee el request, llama a la función y escribe la respuesta tal cual. Esta separación aporta tres ventajas:

- **Pruebas sin red:** la función se puede ejercitar offline y comparar contra las observaciones capturadas.
- **Control byte a byte:** se controla el orden y el uso de mayúsculas de los headers, el cuerpo exacto y el código de estado.
- **Reinicio trivial:** `POST /__reset` equivale a reinicializar la estructura de estado.

Se evita deliberadamente el uso de los valores por defecto de frameworks como FastAPI o uvicorn (respuestas 422 de validación, 404 propios, headers añadidos automáticamente), porque introducen comportamientos del framework que no coinciden con los del original. Se replican los errores del servicio original, no los del framework.

## 3. Método forense

El entendimiento del servicio se construye con un ciclo iterativo de experimentación:

- **Captura:** un arnés propio registra cada par request/response en crudo, en formato JSONL, con los cuerpos codificados en base64 y los headers con su orden y mayúsculas reales. Este registro es, además,

la evidencia que cita la especificación.

- **Fuzzing por capas,** de la envoltura hacia dentro:
  - L0 — Transporte y framing: métodos aceptados, headers de respuesta, trailing slash, codificación, manejo de cuerpos mal formados.
  - L1 — Autenticación: sin header, formato inválido, token incorrecto y token correcto.
  - L2 — Semántica: qué hace realmente el servicio en su camino principal.
  - L3 — Estado: dependencia del orden, acumuladores, idempotencia.
  - L4 — Bordes y errores: entradas vacías, enormes, unicode, valores límite y los bugs del original.
- **Inferencia de reglas:** a partir de lotes de observaciones se deducen las reglas que las explican (apoyándose en un LLM para detectar patrones) y se implementan en `respond()`.
- **Prueba diferencial:** se envía el mismo input al original y al reemplazo y se comparan los bytes tras una normalización controlada, iterando hasta que la divergencia sea prácticamente nula.

El objetivo es inferir la regla general, no memorizar pares entrada/salida: el conjunto oculto usará entradas que no hemos probado.

## 4. Modelado de estado y /__reset

El servicio mantiene estado entre requests —de ahí la existencia de `/__reset`—, de modo que el reemplazo lo modela de forma explícita. `POST /__reset` solo reinicia el estado cuando llega con su token; con un token inválido o sin token, responde como lo haría el original ante esa ruta, replicando su comportamiento de autenticación. Para clasificar el estado se comparan secuencias del tipo `reset → mutar → observar` frente a `reset → observar`, y se determina si un eventual límite de tasa se reinicia con `/__reset` o depende de una ventana de tiempo.

## 5. Riesgos y decisiones de ingeniería

- **Compatibilidad con los errores:** se replican los comportamientos incorrectos del original en lugar de corregirlos; el objetivo es un clon fiel, no un servicio "mejor".
- **Límite de tasa:** el arnés de fuzzing respeta el límite con un paso serializado y detecta cuándo una respuesta es fruto de throttling, para no contaminar el modelo. Si el original responde 429, ese comportamiento se modela como parte del estado reiniciable, con su forma exacta (código, headers como `Retry-After`, cuerpo).
- **No determinismo:** cuando las respuestas incluyen timestamps o identificadores aleatorios, una función `normalize()` los canoniza antes de comparar.
- **Conjunto oculto limitado a tres corridas:** la verificación exhaustiva se hace localmente con la prueba diferencial antes de gastar una corrida; se reserva al menos una para después de un posible cambio a mitad de jornada.
- **Cambios en caliente:** el diseño modular permite que un cambio de comportamiento se traduzca en una sola función a ajustar y volver a verificar; el log, que es append-only con marca de tiempo, deja registrado el cambio de régimen.

## 6. Arquitectura de despliegue

El entrypoint se despliega en un host real (Railway), no en un túnel ni en un equipo local: si el servicio se cae durante el debrief, el conjunto de evaluación no puede ejecutarse. Expone `GET /health` para indicar que está listo, exige `Authorization: Bearer` en todas las llamadas, atiende la ruta base y cualquier otra (`/*`) replicando rutas y errores, y ofrece `POST /__reset` para volver al estado inicial. En ningún caso el reemplazo contacta al servicio original.

## 7. Entregables

- Reemplazo desplegado y accesible desde internet.
- Especificación del servicio: cada comportamiento descrito cita la línea del log que lo demuestra.
- Log de experimentos.
- Video de 2 minutos.

Restricciones respetadas: no se usan browser-agents ni herramientas que hagan el trabajo por nosotros —solo scripts y fuzzers propios—, y el reemplazo no llama al servicio original ni a ningún otro reemplazo.